How Programs Are Built and Run

Video 1 · From Source Code to a Running Program

How Computers Work · Section 8

In this video, you will learn:

- How source code becomes a running program
- Compiled vs interpreted languages
- The build pipeline: compile and link

Source code must be translated

People write **source code**: readable program text (C, Python, JavaScript). But source is just **text**, and the CPU runs only **machine code**.

```c
int main(void) {
    printf("Mina Coffee\n");
    return 0;
}
```

Readable to us, unrunnable by the CPU. Something must translate it. Two ways: **compiling** and **interpreting**.

Compiled vs interpreted

- **Compiled** (C, C++, Rust, Go): translated **ahead of time** into an **executable** of machine code. Ship the executable, not the source.
- **Interpreted** (Python, JavaScript): translated **as it runs**, line by line, by an **interpreter**. Ship the source.

For analysis:

- Compiled → logic hidden in machine code → **reverse engineer** it (a mystery `.exe`).
- Interpreted → logic shipped as **readable source** (easy-to-write malicious scripts).

The build pipeline

```
$ clang -o greet greet.c
$ ./greet
Mina Coffee
2 lattes = 2 coffees
```

That one command hides two stages:

1. **Compile:** source → machine code (**object files**), for *your* code.
2. **Link:** join your object files + **library** code (like `printf`) into one **executable**.

source → compile → object files → link → executable. Reversing runs this **backward**.

Recap: source to program

- The CPU runs **machine code**, not source; source must be **compiled** or **interpreted**.
- **Compiled** hides logic in machine code (reverse it); **interpreted** ships readable source.
- Building = **compile** (your code → machine code) + **link** (join with libraries).

Next video: machine code, libraries, and executable files.

By TOUCH DEN · November 11, 2026

How Programs Are Built and Run

Video 2 · Machine Code, Libraries, and Executables

How Computers Work · Section 8

In this video, you will learn:

- Machine code and assembly
- Static and dynamic libraries
- Executable formats (ELF, PE, Mach-O), and why `strings` matters

Machine code and assembly

The compiler produces **machine code**: raw numeric instructions the CPU runs, just bytes, unreadable to humans.

Its readable form is **assembly**: each instruction as a short mnemonic, `add`, `mov`, `cmp`, `jmp`.

The ladder:

- **Source code** (C, high level) →
- **Machine code** (bytes the CPU runs) ↔
- **Assembly** (readable form of machine code).

Compiler goes **down**; a reverse engineer climbs **back up** (machine code → assembly) with a **disassembler**. (Course 17.)

Libraries: static and dynamic

Our program used `printf`, which we never wrote, it came from a **library** (reused code). Most of a program is library code.

- **Static linking:** library copied **into** the executable (self-contained, bigger).
- **Dynamic linking:** library stays separate (`.dll`, `.so`, `.dylib`), loaded at run time (shared, updatable).

Security: a library flaw is your flaw (**supply-chain risk**); dynamic loading enables **library hijacking**.

Executables, and why `strings` matters

An executable = machine code **+** info the OS needs to run it. Format by OS: **ELF** (Linux), **PE/.exe** (Windows), **Mach-O** (macOS).

```
$ file greet
greet: Mach-O 64-bit executable arm64
$ strings greet | grep -i mina
Mina Coffee
```

Even in compiled code, text strings survive. `strings` on an unknown program is a classic **first step** in malware analysis, URLs, messages, and clues sit in plain sight.

Recap: machine code to malware analysis

- **Machine code** = bytes the CPU runs; **assembly** = its readable form (reverse engineers read this).
- **Libraries** (static/dynamic) are most of a program, supply-chain risk, hijacking.
- Executables are structured files: **ELF / PE / Mach-O**; `strings` and `file` are fast analysis wins.
- Reversing and malware analysis **run this whole journey backward**.

Next lesson: how a computer goes from power-off to a running OS, the boot process.